SKN26_3rd_PROJECT_1TEAM

RAG 기반 AI 자기소개서 첨삭 서비스

JobPocket

조라에몽의 만능 도구들 · 이창우, 장한재, 전종혁, 조동휘, 최수아, 홍완기 · 2026.04.27

AGENDA

진행 순서

01 **프로젝트 배경 & 문제 정의** 기존 AI 취업 서비스 조사 · 해결 방향

02 **데이터 & 전처리 파이프라인** HuggingFace 7.4만 건 · 스키마 · 전처리

03 **시스템 아키텍처 & 기술 스택** 4-Layer Hybrid · Stack · Deploy

04 **RAG 파이프라인 & 알고리즘** Hybrid Retrieval · 초안생성/수정 로직

05 **시연 영상** 실제 시연 설명

06 **테스트 결과 & RAG 품질 평가** pytest · 골든셋 · Retrieval 실측

07 **개발 계획 & 향후 확장 방향** 스프린트 · 트러블슈팅 · 로드맵

프로젝트 배경 & 문제 정의

기존 AI 취업 서비스 현황

- ✓ 자소서 초안 생성·첨삭, 모의면접 제공
- ✓ 에이전트 형태로 진화 중 (Agent 지향)
- ! 단방향 출력 구조 · 사용자 통제권 수준 낮음
- ! 전체 데이터 무차별 활용 · 노이즈 발생

도출된 핵심 문제점

01
블랙박스식 출력

주관적 피드백으로 신뢰성 부족
사용자가 결과에 개입 어려운 단방향 구조

02
저품질 데이터 혼입

합격 기준 미달 데이터까지 학습에 포함
→ 답변 품질 저하

03
소프트 스킬 부재

협업·문제해결 등 역량 발굴 기능이 미흡
STAR 기반 서술로 연결되지 않음

해결 방향 및 기대 효과

해결 방향 및 전략

데이터 기반 품질 필터링

7.4만 건

이력서 데이터 분석

high 등급 기준
selfintro_grade = 'high' 고품질 데이터

RAG 파이프라인 적용 전략

- ★ high 등급 데이터만 선별 → RAG 답변 품질 극대화
- ★ 기술 스킬 기반 벡터 검색 (Qwen3-Embedding 1024차원)
- ★ Hybrid 아키텍처 - 초안은 로컬, 첨삭은 API로 분리
- ★ 문항 유형 자동 분류 → STAR 기법 최적화

기대 효과

01

데이터 기반 객관적 피드백

문법 교정·형식 맞춤 수준을 넘어서는
정량 지표 기반의 첨삭 피드백 제공

02

소프트 스킬 발굴 결합

협업·문제 해결·리더십 등 경험에서
소프트 스킬을 자동으로 발굴해 서술 반영

03

고품질 자소서 첨삭 경험

직무 적합도 + 기술·역량 균형을
통합 반영한 합격 지향 자소서 생성

수집 데이터 개요

HuggingFace · Youseff1987/resume-matching-dataset-v2

jobpost string · lengths 570 1.11k	resume_grade string · classes 3 values	selfintro_grade string · classes 3 values	resume string · lengths 308 1.77k	selfintro string · lengths 824 3.43k	evaluation string · lengths 622 1.33k	total_score int64 13 100	resume_score int64 10 50	selfintro_score int64 0 61
기업명: [코끼리 테크놀로지스] **포지션명**: [백…	상	중	**[이력서]** - 이름: 나무빛 - 생년월일: 1998년 5월 14일 (만 26…	**1) 자기소개: 진정성과 열정을 가지고 나아가다** 안…	<result> <total_score>85</total_score> <resume_score>45</resume_score>…	85	45	40

데이터 규모

74,147
총 레코드 수

2.0만
high 등급 선별

high
필터링 기준

데이터 분할

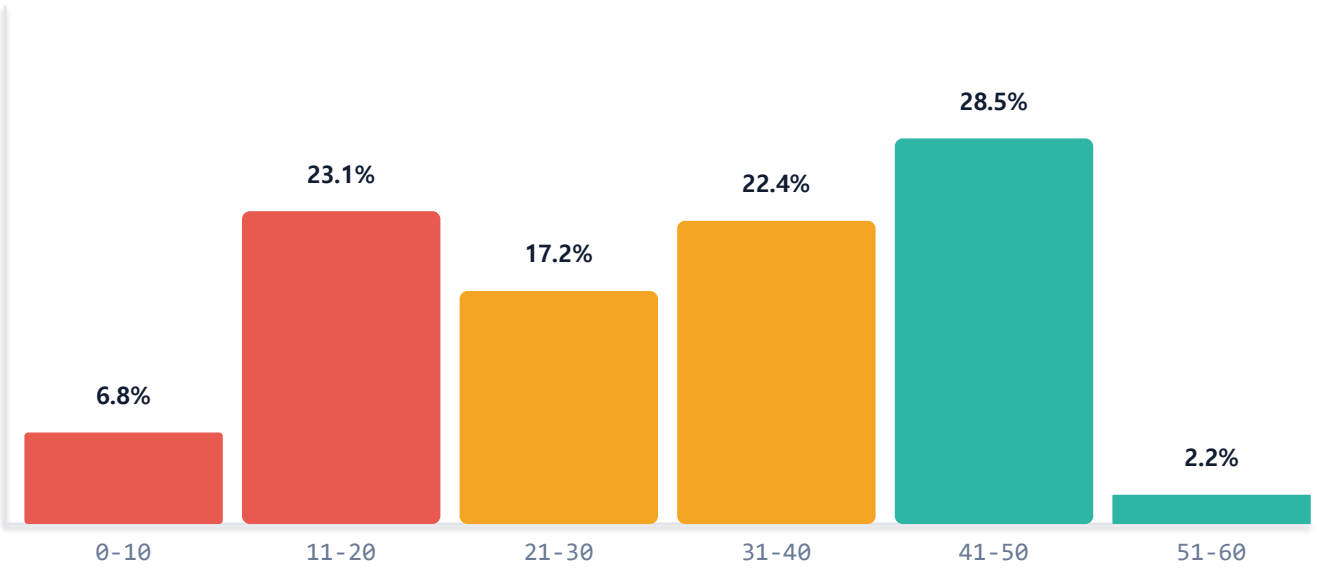
train	69,000 rows	학습·벡터 인덱스 구축
validation	100 rows	모델·파이프라인 검증
test	5,000 rows	최종 평가 및 골든셋

데이터셋 특성

- ✓ **이력서 + 자소서 DATA**
  합격자 자소서 + 평가 포함
- ✓ **직무 대분류**
  Backend · Frontend · AI Engineer
- ✓ **등급 기반 품질 분류**
  high / mid / low 등급 분류
- ✓ **STAR 구조 학습 소스**
  레퍼런스로 고득점 패턴 추출

selfintro_score 분포 & 데이터 분할

selfintro_score 분포 (0 ~ 60)



관찰 결과

selfintro_score는 41-50 정점(28.5%) · 51-60 급락(2.2%)의 종형 분포

참고: 적재본(20,651건) 내부에서는 41-50점 자소서가 88.7%로 집중

RAG 적재 기준은 selfintro_grade = 'high' → 20,651건

Train / Val / Test 분할

train 학습 · 벡터 인덱스 구축	69,047 rows	93.1%
validation 모델 · 파이프라인 검증	100 rows	0.1%
test 최종 평가 · 골든셋	5,000 rows	6.7%

※ 최종 RAG 적재 기준

```
selfintro_grade = 'high'
+ dropna(how='any')
```

데이터 스키마 설계 — HF → MySQL → FAISS

high 필터

id 참조

HuggingFace Dataset

74,147건	총 레코드
jobpost · resume	원본 필드
selfintro · evaluation	자소서 & 평가
selfintro_score	0 ~ 60
selfintro_grade	high / mid / low (원본 상/중/하)

→

MySQL · applicant_records

id	BIGINT PK
selfintro	TEXT NOT NULL
selfintro_evaluation	TEXT
selfintro_score	INT (30~60)
grade / career_type	ENUM

→

FAISS · index.pkl Document

id	UUID
page_content	MySQL record id (본문 ×)
metadata	{ selfintro_score }
embedding	Qwen3 1024-dim
normalized	L2 = 1

핵심 설계

FAISS page_content = MySQL record id → 인덱스 크기 최소화 + 데이터 갱신 유연성 · 상위 id에만 IN 쿼리로 본문 페치

전처리 파이프라인 6-STEP

STEP 1
raw_dataset
입력
HuggingFace 74,147건 로드

STEP 2
채용공고
추출·정제
company · position_type · career_type

STEP 3
이력서
정제
resume_cleaned
텍스트 표준화

STEP 4
자소서 & 평가
추출
selfintro · selfintro_evaluation

STEP 5
결측치 제거
필터링
dropna(how='any') · grade=high

STEP 6
최종
DataFrame
적재 준비된 7필드 구조

최종 출력 컬럼 (7필드)

- `company`
- `position_type`
- `career_type`
- `resume_cleaned`
- `selfintro`
- `selfintro_evaluation`
- `selfintro_score`

dropna(how='any') → selfintro_grade == 'high' 만 최종 적재

OVERSTATEMENT_PATTERNS (9종 금지어)

차별화된 경쟁력을 확보	사회적 영향력을 확대	혁신을 선도
업계를 선도	압도적인 성과	무궁한 발전
최고의 인재	실현하겠습니다	주도하겠습니다

전체 시스템 아키텍처 (4-Layer Hybrid)

USER

Frontend Layer

Streamlit Frontend

- 기술스택 / 직무 입력
- 자소서 출력 UI
- 채팅 인터페이스

Backend Layer

FastAPI Backend

- API 키 보호 (외부 API 격리)
- 비동기 I/O 처리
- HTTP / REST

RunPod Serverless

Iceland Region
RTX 5090 GPU

EXAONE-3.5-7.8B
전용 추론 엔드포인트
Container 기반 배포

AI / RAG Layer

Embedding

Qwen3-0.6B · 1024차원
정규화 벡터 사용

FAISS Vector Search

코사인 유사도
top_k = 3

Open Source LLM

EXAONE-3.5-7.8B
STAR 초안 생성 · 비용 0원

External APIs

Groq API (gpt-oss-120b)
OpenAI (gpt-4o-mini)
→ Backend만 통신

Data Layer

MySQL 9.3

VECTOR(1024) · chat / user
RAG 레퍼런스 DB

FAISS Index

in-memory · page_content = user_id
경량화 파일 관리

HuggingFace Dataset

resume-matching-v2 · 7.4만 건
high 등급만 적재 (약 2.0만 건)

Performance

Groq 사용 시 비용 0원
로컬 임베딩 → 비용 X

Layer 1 & 2 — Frontend / Backend

Streamlit (Frontend)

채팅 기반 UI
사용자의 기술 스택, 활동 경험, 직무 입력
인터랙티브 대화 인터페이스

자소서 출력
STAR 구조 초안 스트리밍 출력
실시간 생성 표시

빠른 배포
프로토타이핑 & 프로덕션
동일 환경에서 운영

HTTP → FastAPI
RESTful API 호출
모든 연산은 Backend 위임

FastAPI (Backend)

비동기 I/O
Python async/await
AI API 통신 & DB 쿼리 동시 처리

API 키 보호
모든 외부 API(Groq, OpenAI)
Backend에서만 호출 — 키 노출 방지

MySQL 세션 관리
유저 정보 & 채팅 히스토리 관리
RAG 레퍼런스 쿼리 수행

RESTful 엔드포인트
단계별 AI 처리 엔드포인트

RunPod + Docker + CI/CD

RunPod 클라우드
Region: 아이슬란드 (Iceland)
GPU: RTX 5090 · 고성능 AI 추론

Docker 컨테이너
Frontend / Backend / DB
각 서비스 독립 컨테이너

CI/CD 자동화
코드 Push → 자동 빌드·테스트
GitHub Actions 롤링 업데이트

환경 일관성
dev / staging / prod 동일
배포 재현성 보장

Layer 3 — RAG Pipeline (AI 엔진)

01 입력 수신

사용자 기술 스택 & 직무
FastAPI를 통해 RAG 엔진으로 전달

02 임베딩 생성

Qwen3-embedding-0.6B
1024차원 임베딩 벡터 생성
사용자 입력 & DB 데이터 모두 적용

03 FAISS 벡터 검색

in-memory FAISS 인덱스
코사인 유사도 기반 Top-3 추출
page_content = user_id (경량화)

04 필터링 & 재정렬

MySQL DB 조회
user_id → 실제 자소서 레퍼런스 반환

05 LLM 생성 (EXAONE)

EXAONE-3.5-7.8B (RunPod flash)
RAG 레퍼런스 3건 컨텍스트 주입
STAR 구조 자소서 초안 생성
품질 점검 후 최대 3회 재시도

06 응답 반환

생성 초안 응답
FastAPI → Streamlit
사용자에게 생성 결과 표시

Layer 4 — Data Layer

MySQL 9.3

VECTOR 타입 (1024차원)

MySQL Community: VECTOR type 존재
벡터 계산 함수 미제공
→ FAISS 보완 도입

유저 & 세션 관리

사용자 계정, 스펙 데이터 저장
채팅 히스토리 저장
RAG 레퍼런스 원본 데이터 적재

DISTANCE() 함수

내장 VECTOR 유사도 함수 제한적
→ in-memory FAISS로 대체
Top-N 추출 후 MySQL 재조회

벡터 테이블 → FAISS

DB 벡터 로드 후 FAISS index
파일 생성 스크립트 구현
주기적 동기화 가능

FAISS Vector Index

in-memory 인덱스

서비스 시작 시 FAISS 인덱스
파일 메모리에 로드
디스크 I/O 없이 초고속 검색

경량화 설계

page_content = user_id
실제 문서 대신 ID만 저장
파일 크기 최소화 (23KB)

코사인 유사도 검색

top_k = 3건 LLM 전달
1024차원 정규화 벡터

MySQL 재조회

FAISS 결과 user_id 리스트
→ MySQL SELECT로 원본 데이터
벡터-문서 분리 패턴

HuggingFace Dataset

Youseff1987/resume-matching-v2

총 7.4만 건 이력서 데이터
직무 대분류 통합 매핑
등급 기반 품질 분류

Performance Targets

0원	LLM 생성 비용 (Groq)
20,000+	RAG 레퍼런스 건수
high grade	고품질 데이터만 적재

기술 스택 · Docker Compose 배포

Backend

Python	3.12	**FastAPI**	0.115+
LangChain	1.2.2+	**PyMySQL**	Engine Pool

AI / ML

EXAONE 3.5	7.8B · RunPod	**GPT-4o-mini**	OpenAI · 첨삭
GPT-OSS-120B	Groq · 첨삭	**Qwen3-Embed**	0.6B · 1024d

Data / Infra

MySQL 9.3	RDB + VECTOR	**FAISS 1.12+**	CPU · 1024d
Docker	3컨테이너		

Test / Ops

pytest	markers · 5종	**LangSmith**	관측 · 선택
GitHub Actions	CI/CD 자동화	**RunPod Serverless**	runpod flash

Docker Compose 토폴로지

frontend
Streamlit
`port 8501 → 8501`

→

backend
FastAPI + RAG
`port 8000 → 8000`

→

database
MySQL 9.3
`port 3306 → 3306`

초안 생성 & 수정 알고리즘 — 이원화 설계

"초안 생성"과 "수정"을 분리: 비용 큰 장문 생성은 로컬(EXAONE), 품질 중요한 후처리·수정만 API(GPT-4o-mini/Groq)

DRAFT · 초안 생성 (Local · EXAONE)

- 비교적 긴 본문을 1회 장문 생성 → API 비용 大
- 로컬 모델로 처리 시 API 호출 비용 0원
- EXAONE 3.5 7.8B · RunPod Serverless (workers 0→1)

```
FUNCTION generate_draft(user, req):
  parsed = parse_user_request(req)     # 회사/직무/문항
  samples = retrieve_top3(parsed)      # FAISS → MySQL
  style = extract_style_rules(samples) # 패턴만 참고
  draft = EXAONE.generate(prompt, user, style)
  WHILE not score_local_draft(draft):  # 최대 3회
    draft = EXAONE.generate(prompt + reason)
  RETURN clean(draft)
```

REVISE · 수정 & 첨삭 (API)

- 기존 초안 기준 부분 수정 → 입력/출력 짧음
- 품질이 중요한 문장 보정만 API로 위임
- GPT-4o-mini (기본) / GPT-OSS-120B (Groq, 사용자 선택)

```
FUNCTION revise(existing_draft, request):
  # 전체 재생성 대신 기존 본문을 기준으로 반영
  revised = API.revise(existing_draft, request)
  refined = API.refine(revised)         # 자연스럽게
  fitted  = fit_length_if_needed(refined, target)
  eval_t  = evaluate_draft_with_api(fitted)
  RETURN build_final_response(fitted, eval_t)
```

LOOP

생성 → 평가 → 보완 포인트 → 수정안 생성

핵심 구현 코드 — 유사 자소서 검색

01
쿼리 입력
사용자 이력서
프로필 기반 쿼리

02
FAISS 검색
코사인 유사도
Top-K 후보 추출

03
ID 추출
page_content
= DB record id

04
MySQL 조회
IN 쿼리로
자소서 본문 페치

05
Document 조립
LangChain Doc
→ LLM 컨텍스트

핵심 설계 포인트

★ FAISS에는 본문 대신 id만 저장 → 인덱스 크기 최소화 (23KB)

★ IN 쿼리로 한 번에 여러 레코드 조회 → DB 왕복 최소화

★ 유사도 순서(db_ids) 기준으로 최종 결과 재정렬

retrieval_schemas.py – 데이터 구조 정의

```python
class RetrievalResult(BaseModel):
    id: int = Field(...,
        description="DB 레코드 ID")
    content: str = Field(...,
        description="자소서 본문")
    selfintro_score: int = Field(0)
    relevance_score: float = Field(0.0)

class RetrievalResponse(BaseModel):
    query: str
    top_k: int
    results: List[RetrievalResult]
```

01

schemas의 역할

검색 결과가 어떤 모양이어야 하는지 Pydantic으로 명확하게 정의. 코드만 봐도 어떤 데이터가 오가는지 파악 가능

02

RetrievalResult

검색 결과 1건 = id + 자소서 본문 + 자소서 평가점수 + 벡터 유사도 점수

03

RetrievalResponse

전체 응답 구조. 쿼리, top_k, 결과 리스트를 묶어서 API 응답·테스트에 바로 활용

04

스코어 이중 관리

자소서 평가 점수(DB)와 벡터 유사도 점수(FAISS), 두 점수를 함께 보존해 활용 가능

retrieval_repository.py – DB 조회

```python
def get_applicant_records_by_ids(
    db_ids: List[int]
) -> Dict[int, Dict]:

    raw_conn = vector_engine.raw_connection()
    try:
        with raw_conn.cursor(DictCursor) as c:
            format_strings = ",".join(["%s"] * len(db_ids))
            sql = f"""
                SELECT id, selfintro
                FROM applicant_records
                WHERE id IN ({format_strings})
            """
            c.execute(sql, tuple(db_ids))
            rows = c.fetchall()
            return {row["id"]: row for row in rows}
    finally:
        raw_conn.close()
```

DB 조회만 담당

SQL 로직이 이 파일 하나에 집중
나중에 ORM 전환·캐싱 추가 시 이 파일만 수정하면 됨

IN 쿼리 일괄 조회

id 목록을 받아 DB 왕복 1회로 여러 레코드를 한꺼번에 가져옴
효율적인 배치 조회

파라미터 바인딩

format_strings = ",".join(["%s"] * N) 방식으로 SQL 인젝션 방지

dict 형태로 반환

{ id: row } 딕셔너리로 반환해서 service에서 O(1) 조회 가능

finally 보장

에러 발생 여부와 무관하게 DB 커넥션을 반드시 반납
커넥션 누수 방지

retrieval_service.py – 검색 총괄

```
def search(self, query: str) -> List[Document]:
    # ① FAISS 유사도 검색
    docs_and_scores = self.vectorstore.similarity_search_with_score(query, k=self.top_k)

    # ② ID + 점수 추출
    for doc, score in docs_and_scores:
        db_id = int(doc.page_content)  # page_content = DB id
        score_map[db_id] = {"selfintro_score": ..., "relevance_score": ...}

    # ③ Repository로 자소서 본문 조회
    record_map = get_applicant_records_by_ids(db_ids)

    # ④ Document 객체 조립 (유사도 순서 유지)
    for db_id in db_ids:  # FAISS 유사도 순서 기준
        doc = Document(page_content=record["selfintro"], metadata={...})


def search_as_schema(self, query: str) -> List[RetrievalResult]:
        docs = self.search(query)
        return [
            RetrievalResult(
                id=doc.metadata["id"],
                content=doc.page_content,
                selfintro_score=doc.metadata["selfintro_score"],
                relevance_score=doc.metadata["relevance_score"],
            )
            for doc in docs
        ]
```

`search( )`

→ `List[Document]`
LangChain 체인에 바로 연결
chat_logic.py AI 파이프라인용

`search_as_schema( )`

→ `List[RetrievalResult]`
Pydantic 스키마로 반환
API 응답 · 테스트에 바로 활용

핵심 구현 코드 — 자소서 생성

parser.py	analyzer.py	generator.py	evaluator.py	run_exaone.py
요청/프로필 파싱	유사 샘플 분석	초안 생성·수정	품질 평가·조립	RunPod 추론

chat_logic.py — 오케스트레이터 & 재생성 루프

```
# [최적화] 샘플 검색은 루프 밖에서 단 1회만 수행
sample = chat_internal.get_sample_context(profile, retrieval_svc, active_llm)

for attempt in range(max_attempts):
    draft = chat_internal.build_draft_with_exaone(current_parsed, profile, sample)
    is_ok, reason = chat_internal.score_local_draft(draft, current_parsed)
    if is_ok: return draft

    # [최적화] 파서 재호출 없이 보완 지시사항만 추가
    current_parsed.raw_message += f"[보완 요청]: {reason} 문제를 해결하여 다시 작성"
```

parser.py — 요청 파싱 (Regex + LLM 2단계 전략)

```python
def parse_user_request(user_message, active_llm):

    # ① 정규식으로 먼저 빠르게 파싱 (비용 0원)
    base = parse_user_request_regex(user_message)

    # ② 정보 부족할 때만 LLM 호출
    if not base.company or not base.job:
        llm_data = llm_parse_user_request(
            user_message, active_llm)

        # 빈 값만 LLM 결과로 채움
        if not base.company:
            base.company = llm_data.get("company", "")

    # JsonOutputParser → ParsedUserRequest 반환
    return base
```

01

Regex 우선 전략

정규식으로 회사명·직무·글자수 빠르게 추출
API 호출 없이 처리 가능한 경우 비용 0원

02

LLM은 보완 역할

정규식이 놓친 경우에만 JsonOutputParser로 LLM 호출
무분별한 API 중복 호출 방지

03

ParsedUserRequest 스키마

company / job / question / char_limit / question_type 5개 필드
파이프라인 전체에 공유

04

문항 유형 자동 분류

motivation / future_goal / collaboration / problem_solving / growth / general 6종 자동 분류

analyzer.py — 유사 자소서 검색 & 패턴 분석

① RetrievalService
유사 샘플 검색

② JsonOutputParser
패턴 요약

③ 작성 규칙 4개
추출

→ SampleAnalysis(summary, style_rules, excerpt) 반환 — chat_logic에서 루프 밖 1회 호출

```
def get_sample_context(profile, retriever, active_llm):
    # ① 사용자 프로필 기반 쿼리로 유사 자소서 검색
    samples = retrieve_raw_samples(query, retriever)

    # ② JsonOutputParser → LLMSampleSummary 스키마로 정밀 요약
    sample_summary = summarize_samples(samples, active_llm)

    # ③ 요약 → 실제 작성에 쓸 수 있는 규칙 4가지로 변환
    style_rules = extract_sample_style_rules(sample_summary, active_llm)

    return SampleAnalysis(summary=sample_summary, style_rules=style_rules, excerpt=excerpt)
```

generator.py — EXAONE 초안 생성 & API 첨삭

build_draft_with_exaone( )

```
# 문항 유형별 시스템 프롬프트 적용
prompt = ChatPromptTemplate.from_messages([
    ("system", get_draft_system_prompt(
        parsed.question_type)),
    ("human", DRAFT_HUMAN_PROMPT_TEMPLATE)
])

# 프롬프트 → dict 변환 → EXAONE 전달
messages = convert_messages_to_dict(
    final_prompt.to_messages())
result = inference_func(messages)

# LLM 출력 헤더·기호 제거
return clean_text(remove_forbidden_headers(result))
```

EXAONE (RunPod GPU) · 초안 창작 담당

refine_with_api( )

```
# EXAONE 초안을 GPT/Groq으로 다듬기
sys_prompt = get_refine_system_prompt(
    parsed.question_type)

result = (prompt | active_llm
          | StrOutputParser()).invoke({...})

return clean_text(remove_forbidden_headers(result))
```

GPT / Groq API · 첨삭·수정·길이조정·평가 담당

LLM 역할 분업

▶ EXAONE : 초안 창작 — 긴 본문 한국어 생성 (RunPod GPU)

▶ GPT/Groq : 첨삭·수정·글자수 조정·평가 수행

evaluator.py — 품질 점검 & 최종 응답 조립

score_local_draft( ) – 규칙 기반 품질 점검

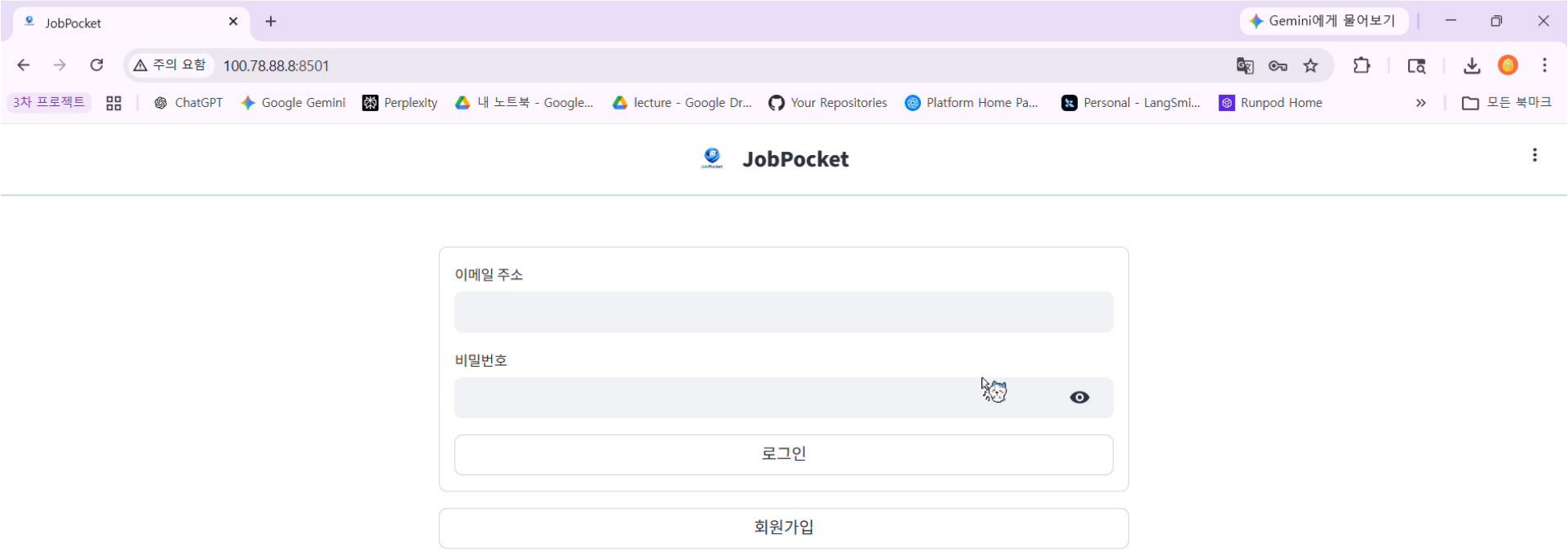
```
if len(text) < 220:
    return False, "너무 짧음"

if repetition_ratio(text) > 0.48:
    return False, "문장 반복 많음"

# 글자수 목표의 55% 미만이면 실패
if current < max(220, int(target * 0.55)):
    return False, "글자수 부족"

# 과장 표현 패턴 검출
for pattern in OVERSTATEMENT_PATTERNS:
    if pattern in text:
        return False, f"과장 표현: {pattern}"

return True, "통과"
```

- ✗ 길이 < 220자
- ✗ 반복 비율 > 48%
- ✗ 목표의 55% 미만
- ✗ 회사명 누락
- ✗ 과장 표현

build_final_response( ) – 최종 조립

```
# JsonOutputParser로 LLM 평가 생성
evaluation = evaluate_draft_with_api(
    body, parsed, active_llm)

# 최종 응답 포맷 조립
lines = [f"[{result_label}]", ""]

if change_summary:
    lines.append(
        f"반영 사항: {change_summary}")

lines.append(body)
lines.append("")
lines.append("[평가 및 코멘트]")
lines.append(evaluation)

return "\n".join(lines).strip()
```

시연 영상


JobPocket
Gemini에게 물어보기
주의 요함
100.78.88.8:8501
3차 프로젝트
ChatGPT
Google Gemini
Perplexity
내 노트북 - Google...
lecture - Google Dr...
Your Repositories
Platform Home Pa...
Personal - LangSmi...
Runpod Home
모든 북마크
JobPocket
이메일 주소
비밀번호
로그인
회원가입

통합 중심 테스트 전략 & 케이스 매트릭스

테스트의 3가지 층위

계획 문서 `test_plan.md`

무엇을 어떤 기준으로 테스트할지 명세

pytest 코드 `backend/tests/`

API 엔드포인트 + 파이프라인 통합 테스트

RAG 품질 평가 `evaluation/`

Retrieval·Generation의 정량 지표 측정

pytest 마커

- `@integration` 통합
- `@api` API
- `@mock_llm` LLM 모킹
- `@retriever` FAISS 필요
- `@requires_llm` 실제 호출

주요 테스트 케이스 (TC-001 ~ 028)

ID	영역	검증 포인트	우선
TC-001~005	인증	로그인·회원가입·비밀번호 재설정	P0
TC-006~008	이력	resume_data JSON 저장·조회	P0
TC-009~017	RAG 파이프라인	Step 1~6 + Revise 경로	P0
TC-018~022	E2E 통합	전체 사용자 시나리오	P0
TC-023~025	Retriever	HybridRetriever + FAISS	P1
TC-026~028	인프라	FAISS 다운로드 · RunPod 연동	P1

CI 자동화 (.github/workflows/ci.yml)

pytest -m "not requires_llm" — LLM 호출 제외 · MySQL 9.3 services 컨테이너 · Mock fixture로 회귀 검출만

RAG 품질 평가 시스템 (evaluation/)

pytest는 "동작하는가"를 답하지만, RAG가 "실제로 좋은가"는 답하지 못한다 → 별도 정량 평가 필요

검색 품질 (Retrieval)

run_retrieval_eval.py — 지원자 스펙과 유사한 이력의 DB 데이터를 잘 찾아내는가?

average TMR	기술 키워드 매칭률 (Tech keywords matching rate)	**전체 테스트셋의 일치율 평균**
Top-n TMR	n번째로 유사도가 높은 문서의 TMR	**순위별 평균**
ZMR	Zero matching rate	**하나도 일치하지 않은 케이스의 비율**

```
$ python evaluation/run_retrieval_eval.py \
    --file TEST_DATASET_NAME.json
-평가 결과는 results/에 저장
```

생성 품질 (LLM-as-Judge)

run_generation_eval.py — 생성된 자소서가 실제로 좋은가? (5기준)

1	**문항 적합성**	질문에 직접 답하는가
2	**직무 적합성**	경험-직무 연결 자연스러움
3	**이력 반영도**	사용자 경험이 구체적으로 녹아있음
4	**첫 문장 완성도**	훅이 되는가
5	**마지막 문단**	마무리가 설득적인가

```
$ python evaluation/run_generation_eval.py \
    --limit 5 --skip-judge    # 저비용 모드
$ python evaluation/run_generation_eval.py    # 전체(~$2)
```

Retrieval 품질 실측 (4,005건)

4,005건 평가 샘플 규모

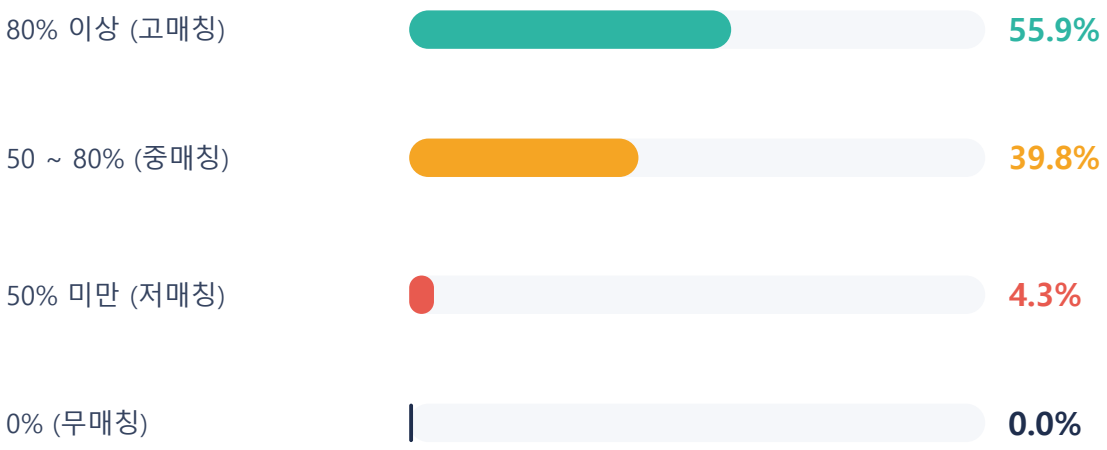
78.9% 전체 평균 매칭률 (Rank 1)

0건 무매칭 케이스 (검색 실패 없음)

직무별 × 순위별 매칭률

직무	Rank 1	Rank 2	Rank 3
Frontend engineer	83.27%	82.19%	81.94%
Backend engineer	78.70%	77.71%	77.28%
AI engineer	73.88%	71.53%	71.71%
TOTAL AVERAGE	78.91%	77.50%	77.32%

Rank 1 매칭률 분포



▶ **핵심 해석**

쿼리당 평균 9개 기술 · Top-3 편차 σ≈0.17 · Frontend(83%) > Backend(79%) > AI Engineer(74%) — AI 직무는 동의어 밀도↑ 영향

개발 계획 — 스프린트 타임라인

1주	1	12
전체 개발 기간	Sprints · 주 단위	Tasks (T-01 ~ T-12)

작업 분배 (WBS + 병렬 그룹)

Sprint 1 (Week 1)

ID	작업명	D1	D2	D3	D4	D5
T-01	개발 환경 구축 (Runpod)					
T-02	RAG 데이터셋 구축 (고득점 자소서)					
T-03	데이터 전처리 (직무 매핑 + MySQL 적재)					
T-04	Qwen3 임베딩 + FAISS 벡터 적재					
T-07	FastAPI 코어 셋업					
T-05	STAR 초안 생성 API (RAG + Groq)					
T-06	초안 개선 루프 API (선택지 재생성)					
T-11	GitHub Actions CI/CD 구축					
T-08	Streamlit 채팅 UI 연동 (STAR 렌더링)					
T-09	Diff UI + 파일 업로드 화면					
T-10	Streamlit 전체 통합					
T-12	통합 테스트 & 배포					

Infra · Backend/ML · Frontend · 통합/전체

트러블슈팅

01 torch packing 실패 → --no-deps 패턴

문제 Flash가 torch wheel(~2GB) 번들링 실패 + --no-deps 단독으로는 deploy 차단

해결 @Endpoint dependencies 비우고 requirements.txt + flash deploy --no-deps 분리 — 표준화

02 CUDA Out of Memory (OOM)

문제 max_new_tokens 과대 또는 누적 토큰 길이 초과 시 KV Cache 폭증으로 Worker 크래시

해결 max_new_tokens 상한(1024) + 응답 후 gc.collect() · torch.cuda.empty_cache()

03 자기소개서 초안 생성 과정 최적화

문제 초안 생성 루프 내에서 중복된 검색 및 api 호출을 통한 내용 추출이 반복 수행되어 토큰 낭비 및 지연 발생

해결 검색/추출 단계를 루프 외부로 분리하여 최초 1회만 수행하고, 재생성이 필요한 경우 재사용하도록 개선

04 @Endpoint 직접 호출 + Pydantic 미인식

문제 FastAPI에서 @Endpoint 함수 import 시 TypeError, Pydantic 입력 스키마 packing 실패

해결 HTTP 호출(httpx)로 전환 + 응답 TypedDict / 입력 dict + **kwargs 우회 (입력 스키마 미해결)

버전 로드맵

DONE

v0.1.0

인프라 완료

Docker Compose 스택

DONE

v0.2.0

통합

부분 연동 · 알려진 이슈

DONE

v0.3.0

안정화

6단계 파이프라인 완성

DONE

v0.4.0

최적화

코드 리팩토링 · 성능 개선

현재

v0.5.0

배포

수강생 대상 실배포

✓ 달성한 것 (v0.1.0 ~ v0.5.0)

- MySQL 9 · FAISS · LangChain 기반 Hybrid RAG 구축
- EXAONE 로컬 + GPT-4o-mini/Groq 이원화 완성
- Retrieval 품질 4,005건 실측 (평균 78.9%)
- bcrypt · CORS 제한 · HTTP · 4-worker 실배포 완료

▸ 향후 확장 방향 (다음 슬라이드 상세)

- 사용자 정보 입력 구조 세분화 — 5필드 분해
- 평가를 규칙 기반 점검과 결합 — 신뢰도 강화
- 비용 최적화 구조 고도화 — 동적 LLM 라우팅
- 결과 화면 → 작성 워크플로우로 확장

향후 확장 방향

01 사용자 정보 입력 구조 세분화

큰 덩어리 텍스트 입력 → 프로젝트명 · 역할 · 문제상황 · 해결방식 · 결과 5필드

- 현재: 경험·수상·기술 스택을 자유 텍스트 입력
- 확장: 구조화된 5필드 입력 폼 도입
- 기대: 모델이 없는 경험을 추론하지 않음 · 구체성·정확성 동시 확보

02 평가를 규칙 기반 점검과 결합

LLM-as-Judge 단독 → 규칙 레이어 + 판정 레이어 이중 검증

- 현재: 문항 적합성 등 5기준 LLM 판정 (주관성)
- 확장: 글자수·과장표현·사실성 위반 등 규칙 기반 사전 점검
- 기대: 평가 재현성 ↑ · 피드백 신뢰도 ↑ · 비용 ↓

03 비용 최적화 구조 고도화

정적 로컬/API 분리 → 문항 유형·입력 길이 기반 동적 LLM 라우팅

- 현재: 초안=로컬(EXAONE), 수정=API(GPT/Groq) 고정 분리
- 확장: 짧은 수정 → 로컬, 긴 초안 → API, 요청별 최적 LLM 자동 선택
- 기대: 동일 품질 유지하며 API 비용 30~50% 추가 절감

04 결과 화면 → 작성 워크플로우로 확장

단일 결과 출력 → 버전 관리 · 부분 수정 · 문체 옵션 기반 작성 도구

- 현재: 생성 결과 1회 출력 중심
- 확장: 수정 이력 비교 · 버전별 저장 · 부분 수정 · 문체(formal/casual) 옵션
- 기대: 단순 생성 도구가 아닌 자소서 작성 플랫폼으로 승격

Thank You

Q & A

Job-Pocket · 조라에몽의 만능 도구들

AI 기반 자기소개서 첨삭 서비스

2026.04.27